

Day 3/100 · DevOps & Cloud Series

Kubernetes Monitoring & Observability

10 tools on one notebook 🖋️

 Prometheus

 Alertmanager

 Grafana

 OpenTelemetry

 Thanos

 Fluent Bit

 Promtail → Alloy

 Loki

 ELK

 Retention & storage

For each: What · Problem · Implement · Flow · Confirm

swipe →

~ Rohith Aerva

Monitoring vs Observability

Know **that** vs
know **why** 🤔

	Monitoring	Observability
Question	Is it broken? (<i>known problems</i>)	Why is it broken? (<i>unknown problems</i>)
Example	"CPU > 90%" alert	"Checkout slow only for UPI users in ap-south-1"
Built on	Dashboards + alerts	Metrics + logs + traces, correlated

The 3 pillars

 **Metrics**

Numbers over time
error rate = 5%

.....

Prometheus, Thanos

 **Logs**

Events as text
"payment timeout id=42"

.....

Fluent Bit, Promtail, Loki, ELK

 **Traces**

One request's journey
payments took 4.8s

.....

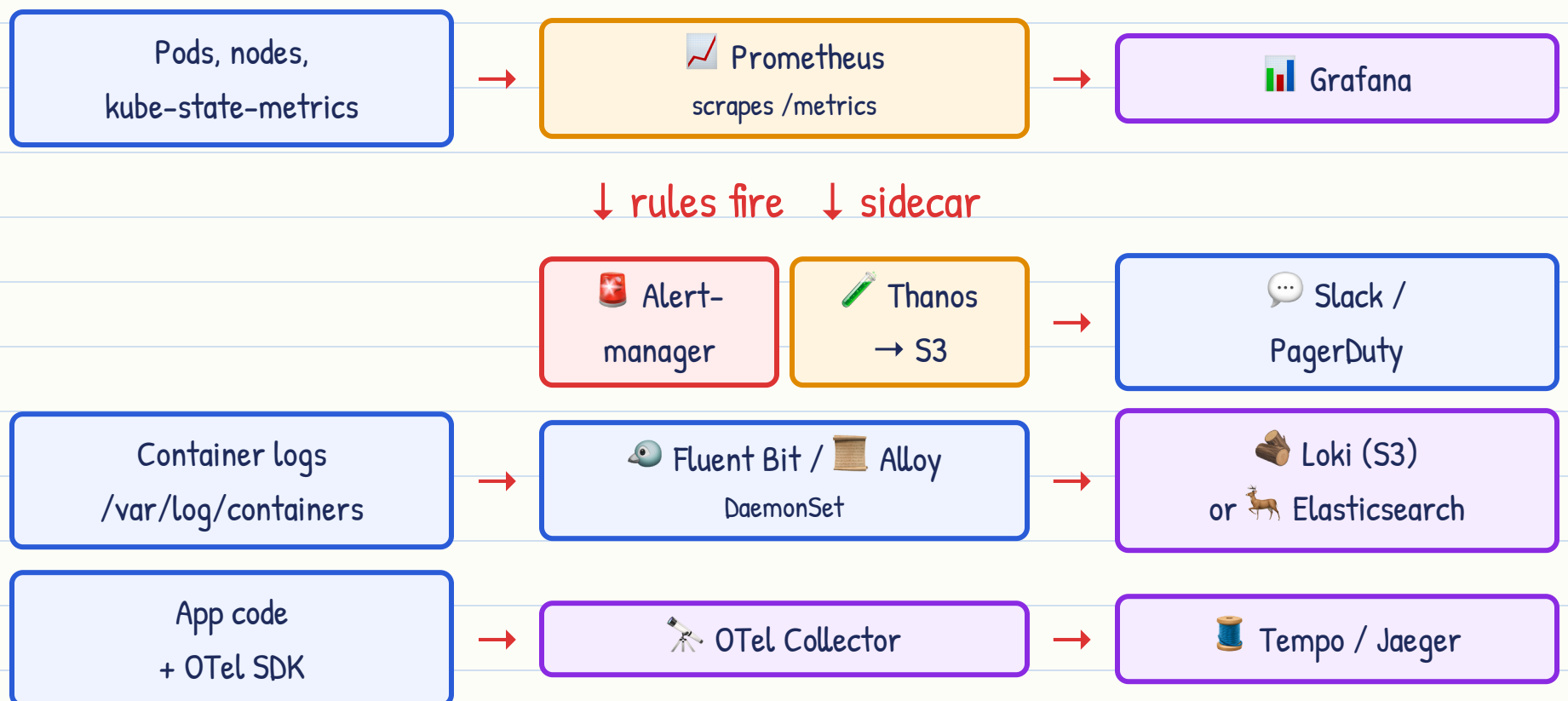
OpenTelemetry, Tempo, Jaeger

Which tool does what

Job	Metrics	Logs	Traces
Collect	Prometheus scrape, OTel	Fluent Bit, Promtail/Alloy, OTel	OTel SDK + Collector
Store	Prometheus TSDB → Thanos (S3)	Loki (S3) or Elasticsearch	Tempo / Jaeger
View	Grafana	Grafana / Kibana	Grafana
Alert	Prometheus rules → Alertmanager → Slack / PagerDuty		

Big picture: how it all connects

The whole picture 🌍



🔗 **Correlation** is the real win: alert fires → open the Grafana dashboard → click the spike → see the logs for that pod → jump to the trace by trace_id → find the slow call.

📦 Usual stack in one namespace

```
kubectl create namespace monitoring
# Prometheus + Alertmanager + Grafana
helm install kps prometheus-community/kube-prometheus-stack -n monitoring
helm install loki grafana/loki -n monitoring
helm install fb fluent/fluent-bit -n monitoring
helm install otel open-telemetry/opentelemetry-operator -n monitoring
```



Prometheus

Metrics engine

What: open-source time-series database + scraper. It **pulls** metrics from `/metrics` endpoints every 15 to 30s and lets you query them with **PromQL**.



Problem it solves

- **X** "Is my cluster healthy?" No numbers
- **X** No history: what was memory at 3am?
- **X** No basis for alerts
- **✓** CPU, memory, restarts, error rate, latency for every pod & node



Flow

node-exporter · kube-state-metrics · app /metrics

↑ scrape (pull) every 30s

Prometheus (TSDB on PVC)

↓ PromQL

Grafana · alert rules



Implement

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
# Scrape your app: ServiceMonitor CRD
kind: ServiceMonitor
spec: {selector: {matchLabels: {app: orders}}, endpoints: [{port: http, path: /metrics}]}
```



Confirm it solved the problem

Check	Expected
<code>kubectl port-forward svc/kps-kube-prometheus-stack-prometheus 9090</code> → Status → Targets	All targets UP
Query <code>up{job="orders"}</code>	Value 1 per pod
<code>sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)</code>	CPU per pod graph

⚠️ Avoid high-cardinality labels (`user_id`, `request_id`): they explode memory.



Alertmanager

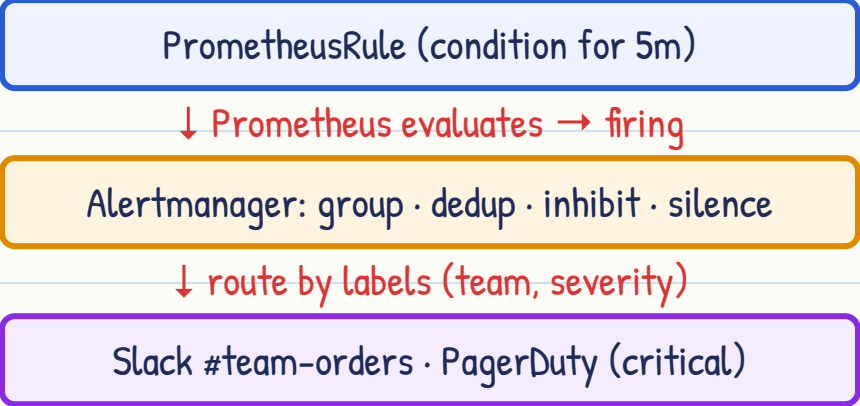
Wake the right person 

What: receives alerts fired by Prometheus rules and decides **who** gets notified, **how**, and **how often**: grouping, dedup, silences, routing.

🤔 Problem it solves

- **X** Nobody watches dashboards at 3am
- **X** 50 pods crash → 50 Slack messages (alert storm)
- **X** Same alert from 2 Prometheus replicas
- **✓** 1 grouped message, routed to the owning team

🔄 Flow



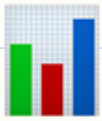
🔧 Implement

```
kind: PrometheusRule
spec:
  groups:
    - name: orders
      rules:
        - alert: OrdersHighErrorRate
          expr: sum(rate(http_requests_total{app="orders",code=~"5.."}[5m])) / sum(rate(http_requests_total{app="orders"}[5m])) > 0.05
          for: 5m
          labels: {severity: critical, team: orders}
```

✅ Confirm it solved the problem

Check	Expected
Deploy a pod with a bad image	KubePodCrashLooping fires
Prometheus UI → Alerts	State: Pending → Firing
Slack / PagerDuty	One grouped message ✓
amtool silence add alertname=X	Notifications stop during maintenance

💡 Alert on **symptoms** users feel (errors, latency), not every CPU spike. Add a runbook link to each alert.



Grafana

One pane
of glass

What: visualization layer. Connects to many **data sources** (Prometheus, Loki, Tempo, Elasticsearch, CloudWatch) and shows dashboards, Explore queries & alerts.

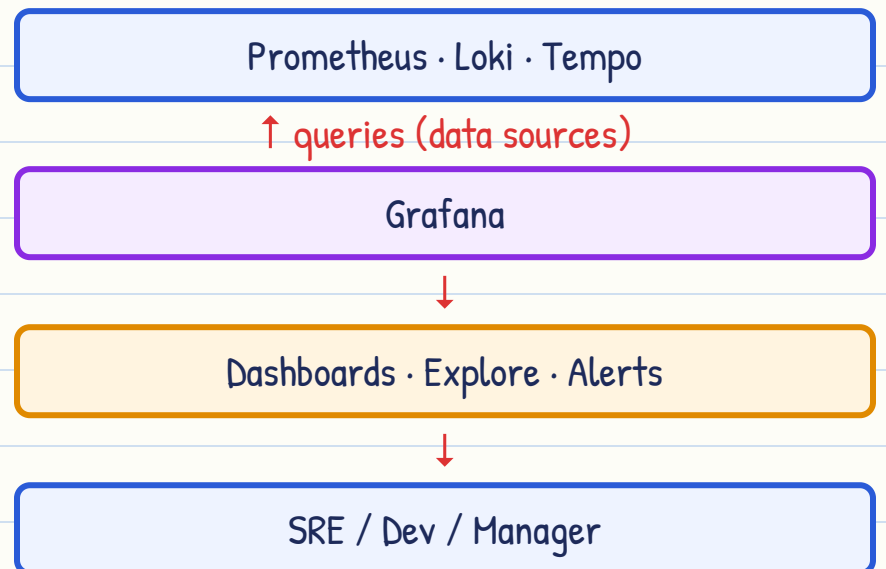


Problem it solves

- **X** PromQL in a terminal is not a dashboard
- **X** Metrics, logs, traces in 3 different UIs
- **X** Each team builds dashboards by hand
- **✓** Shared dashboards, click from metric → logs → trace



Flow



Implement

```
# Installed with kube-prometheus-stack. Dashboards as code:
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-dashboard
  labels: {grafana_dashboard: "1"}      # sidecar auto-loads it
data:
  orders.json: | { ...dashboard JSON... }
```



Confirm it solved the problem

Check	Expected
<code>kubectl port-forward svc/kps-grafana 3000:80</code>	Login page
Connections → Data sources → Test	Successfully queried
Dashboards → Kubernetes / Compute Resources	Live pod CPU/memory
Explore → Loki → {app="orders"}	Logs next to metrics



Use SSO, store dashboards in Git, never edit prod dashboards by hand.



OpenTelemetry (OTel)

One standard,
any backend 🌩

What: CNCF standard + SDKs + **Collector** to generate and ship **traces, metrics and logs** in one vendor-neutral format (OTLP).



Problem it solves

- ❌ Each vendor needs its own agent & SDK
- ❌ Switching Datadog → Grafana = re-instrument
- ❌ No traces: "which service is slow?"
- ✅ Instrument once, send anywhere



Flow

App + OTel SDK / auto-instrumentation

↓ OTLP (4317 gRPC / 4318 HTTP)

OTel Collector

receivers → processors → exporters



Tempo · Jaeger · Prometheus · Loki · X-Ray



Implement

```
helm install otel open-telemetry/opentelemetry-operator -n monitoring
# Auto-instrument without code changes:
kind: Instrumentation
metadata: {name: auto, namespace: shop}
spec: {exporter: {endpoint: http://otel-collector.monitoring:4318}}
# then on the Deployment pod template:
annotations: {instrumentation.opentelemetry.io/inject-java: "true"}
```



Confirm it solved the problem

Check	Expected
Collector logs (debug exporter)	Spans printed
Metric otelcol_receiver_accepted_spans	Increasing
Grafana → Explore → Tempo → search orders	Full trace waterfall
Log line has trace_id	Click log → open trace ✓



Apps must propagate traceparent headers. The SDK does it for you.



Fluent Bit

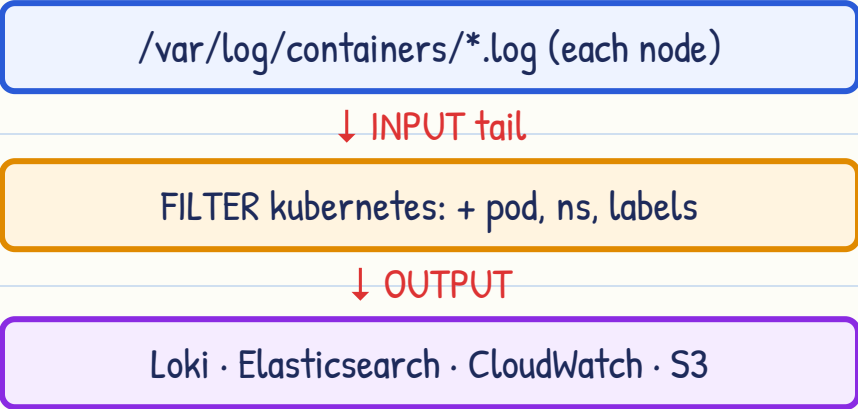
Tiny, fast
log shipper 🚚

What: lightweight log processor (C, ~1 MB) running as a **DaemonSet**. Tails container logs, adds Kubernetes metadata, ships to **any** destination.

🤔 Problem it solves

- ❌ Pod logs die with the pod
- ❌ `kubectl logs` 1 pod at a time
- ❌ Logs lack namespace / pod / labels
- ✅ All logs, enriched, in one central store

🔄 Flow



🔧 Implement

```
helm repo add fluent https://fluent.github.io/helm-charts
helm install fb fluent/fluent-bit -n monitoring
# values.yaml → config.outputs
[OUTPUT]
    Name      loki
    Match     kube.*
    Host      loki-gateway.monitoring
    Labels    job=fluent-bit, namespace=$kubernetes['namespace_name']
```

✅ Confirm it solved the problem

Check	Expected
<code>kubectl get ds -n monitoring</code>	1 pod per node, READY
<code>curl :2020/api/v1/metrics</code>	output <code>proc_records</code> growing
<code>fluentbit_output_retries_failed_total</code>	0
Delete a pod, search its logs	Still searchable ✓

💡 **Fluent Bit vs Fluentd:** Bit = light agent on every node; Fluentd = heavier aggregator with many plugins.



Promtail → Grafana Alloy

⚠ Promtail
is deprecated

What: Promtail was Loki's own log agent (DaemonSet). It reached **end of life on 2 Mar 2026**. Its replacement is **Grafana Alloy**, one agent for logs, metrics & traces (OTel based).



Problem it solves

- ✗ Getting pod logs into Loki with the right labels
- ✗ Running 3 agents for 3 signals
- ✓ Promtail: logs → Loki
- ✓ Alloy: logs + metrics + traces in one agent



Flow

/var/log/pods (each node)

↓ discovery via K8s API

Promtail / Alloy
relabel → namespace, pod, app

↓ push

Loki



Implement

```
# Old (Promtail)
helm install promtail \
  grafana/promtail
```

```
# New (Alloy)
helm install alloy grafana/alloy -n monitoring
# convert existing config:
alloy convert --source-format=promtail \
  --output=config.alloy promtail.yaml
```

```
# Alloy pipeline: discovery.kubernetes → loki.source.kubernetes → loki.write
```



Confirm it solved the problem

Check	Expected
Alloy UI :12345	Components healthy (green)
Promtail :9080/targets (legacy)	Targets ready
Grafana Explore {namespace="shop"}	Logs flowing
After migration	Same labels, no gaps ✓



Interview: "We migrated Promtail to Alloy using `alloy convert` after Promtail went EOL."



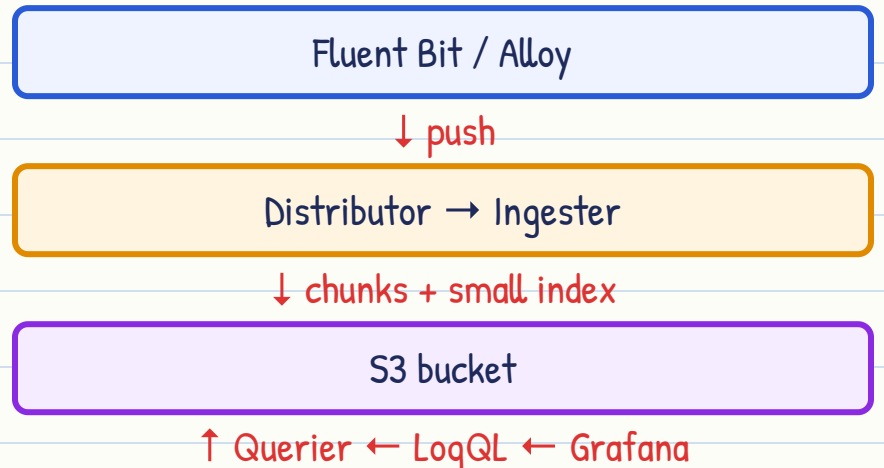
Prometheus,
but for logs

What: log store by Grafana Labs. Indexes only **labels** (namespace, app), not the full text. Log chunks live in cheap **object storage (S3)**.

🤔 Problem it solves

- ✗ Elasticsearch full-text index is costly
- ✗ Logs & metrics use different labels
- ✓ 5 to 10x cheaper storage
- ✓ Same labels as Prometheus → easy correlation

🔄 Flow



🔧 Implement

```
helm install loki grafana/loki -n monitoring -f values.yaml
# values.yaml (important bits)
loki:
  storage: {type: s3, bucketNames: {chunks: acme-loki, ruler: acme-loki}}
  limits_config: {retention_period: 744h}      # 31 days
  compactor: {retention_enabled: true}
```

✅ Confirm it solved the problem

Check	Expected
Grafana Explore: {app="orders"} = "error"	Matching lines
sum(rate({app="orders"} = "error" [5m]))	Error-log rate graph
S3 bucket	chunks/ objects growing
Query 30 days back	Still works ✓

⚠️ Keep labels low-cardinality. Never use pod IP, user_id or request_id as a label.



ELK / EFK Stack

Full-text
search 🔍

What: Elasticsearch (search engine) + Logstash (parse) + Kibana (UI). In K8s usually EFK: Fluent Bit replaces Logstash on the nodes.

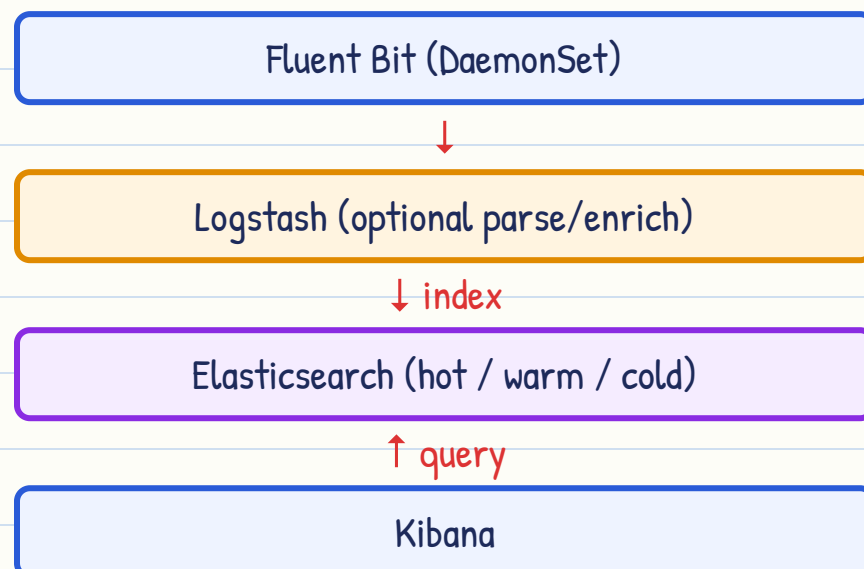


Problem it solves

- ❌ "Find every log containing order 98231"
- ❌ Complex parsing & analytics
- ✅ Full-text search on every word
- ✅ Rich Kibana dashboards, SIEM / security use



Flow



Implement

```
# ECK operator manages ES & Kibana as CRDs
helm install elastic-operator elastic/eck-operator -n elastic-system --create
--namespace
kind: Elasticsearch
spec: {version: 8.x, nodeSets: [{name: default, count: 3}]}
---
kind: Kibana
spec: {version: 8.x, count: 1, elasticsearchRef: {name: logs}}
```



Confirm it solved the problem

Check	Expected
GET _cluster/health	status green
GET _cat/indices?v	Daily log indices growing
Kibana → Discover → search "98231"	Matching logs
ILM policy	Old indices roll to warm, then delete



ELK = powerful search, but needs RAM, disk & ops effort. Loki = cheaper, label-based.



Retention & Storage

Keep enough,
pay less 💰

What: deciding how long to keep data, where it lives, and how to restore it. Without it, disks fill up or bills explode.



Problem it solves

- ❌ Prometheus PVC full → crash
- ❌ Logs kept forever → huge bill
- ❌ Node dies → metrics history gone
- ✅ Hot short-term + cheap long-term + tested restore



Flow

Hot: Prometheus PVC (15 days)



Warm: S3 via Thanos / Loki

↓ lifecycle

Cold: S3 Glacier → delete



Implement

```
# Prometheus (kube-prometheus-stack values)
prometheus.prometheusSpec: {retention: 15d, retentionSize: 45GB,
  storageSpec: {volumeClaimTemplate: {spec: {resources: {requests: {storage:
50Gi}}}}}}
# Loki: limits_config.retention_period: 744h + compactor.retention_enabled
# Elasticsearch: ILM hot 7d → warm 30d → delete 90d
# Backup / restore: S3 versioning, ES snapshots to S3, Velero for PVCs
```



Confirm it solved the problem

Check	Expected
prometheus_tsdb_lowest_timestamp	Oldest data ≈ 15 days ago
kubelet_volume_stats_used_bytes	PVC below 80%, alert set
S3 lifecycle rules	Objects move to Glacier, then expire
Restore drill (ES snapshot / Velero)	Data back in a test cluster ✓



Typical: metrics 15d local + 1 year in S3 · logs 30 to 90 days · audit logs per compliance.



Thanos

Prometheus
superpowers 🦹

What: adds long-term storage (S3), high availability and a global view across many Prometheus servers / clusters.

🤔 Problem it solves

- ❌ Prometheus keeps only ~15 days locally
- ❌ 2 HA replicas = duplicate data
- ❌ 5 clusters = 5 Prometheus UIs
- ✅ 1 query across all clusters & 1+ year

🔄 Flow

Prometheus ×2 (HA) + Thanos Sidecar

↓ uploads 2h blocks

S3 bucket

↑ Store Gateway · Compactor (downsample)

Thanos Query (dedup) ← Grafana

🔧 Implement

```
# kube-prometheus-stack values
prometheus.prometheusSpec:
  replicas: 2
  externalLabels: {cluster: prod-ap-south-1}
  thanos: {objectStorageConfig: {existingSecret: {name: thanos-s3, key: object-store.yml}}}}
# then deploy Query, Store Gateway, Compactor (e.g. bitnami/thanos chart)
# Grafana data source → http://thanos-query:9090
```

✅ Confirm it solved the problem

Check	Expected
Thanos Query UI → Stores	Sidecars + Store Gateway UP
Query 90 days back	Data returned from S3
Toggle "deduplication"	No double lines from 2 replicas
Kill one Prometheus replica	Dashboards still work ✓

🔄 Alternatives: Grafana Mimir, Cortex, VictoriaMetrics, Amazon Managed Prometheus.

Which one should I choose?

Pick the right one 

Loki vs Elasticsearch

	Loki	Elasticsearch (ELK)
Indexes	Labels only	Every word (full text)
Storage	S3, cheap	Disks, RAM heavy
Best for	K8s app logs + Grafana	Search, analytics, SIEM
Ops effort	Low	High

Log agents

Agent	Use when
Fluent Bit	Any destination, very light, CNCF
Grafana Alloy	Grafana stack, one agent for logs + metrics + traces
Promtail	Don't start new: EOL, migrate to Alloy
OTel Collector	Vendor-neutral pipeline for all signals

Long-term metrics

Need	Choose
Single cluster, 15 days	Plain Prometheus
HA + 1 year + multi-cluster, self-managed	Thanos (or Mimir)
AWS, no ops	Amazon Managed Prometheus + Managed Grafana

End-to-end: did we solve it?

Prove it works 

Break something on purpose and time how fast you find the root cause.

#	Chaos test	Must happen ✓	Tool
1	Deploy orders with bad image	Alert in Slack < 5 min	Prometheus + Alertmanager
2	Open alert link	Dashboard shows restarts spike	Grafana
3	Click "logs" for that pod	Error line visible, even after pod deleted	Fluent Bit/Alloy + Loki
4	Inject 3s delay in payments	Trace shows payments span = 3s	OTel + Tempo
5	Query metrics 90 days back	Data returned	Thanos + S3
6	Kill 1 Prometheus replica	No gap in dashboards	Thanos dedup
7	Fill a test PVC to 85%	Disk alert fires	kubelet metrics
8	Restore last snapshot to test cluster	Data back	ES snapshot / Velero

Measure the improvement

Before

MTTD: users tell us 😬

MTTR: 2 to 3 hours

After

MTTD: < 5 min (alert)

MTTR: ~20 min (metric → log → trace)

TL;DR

 Prometheus	Pulls & stores metrics, PromQL
 Alertmanager	Groups, dedups, routes alerts
 Grafana	Dashboards for everything
 OpenTelemetry	Standard to collect traces, metrics, logs
 Fluent Bit	Light log shipper, any destination
 Promtail → Alloy	Loki agent (Promtail EOL, use Alloy)
 Loki	Cheap label-indexed logs on S3
 ELK	Full-text log search & analytics
 Retention	Hot short, cold cheap, restore tested
 Thanos	Long-term, HA, multi-cluster Prometheus

Pro tips

- Use the **same labels** (cluster, namespace, app) across metrics, logs, traces
- Alert on **symptoms** (errors, latency), link a runbook
- Watch **cardinality**: it is the #1 cost & crash cause
- Monitor the monitoring: alert if Prometheus or Loki is down

You can't fix what you can't see 🙄🙄